UNIT – 8
Structure and Union

C Programming (CSC115)
BSc CSIT First Semester (TU)

Prepared by:
Dabbal Singh Mahara
ASCOL (2025)

What is structure?

- A structure is a user-defined data type which collects different types of variables under a single name.
- A structure is a convenient way of grouping several pieces of related information together.
- For example: name, roll, fee, marks, address, gender and phone are related information of a student. Here, name requires an array of characters, roll uses an integer variable, fee, marks uses float type data and so on.
- So, these all information related to a student can be grouped as a single entity in the form of structure.

Defining a Structure

- A structure can be defined as follows:

  Syntax:

```
struct structure_name
{
        data_type          meber_variable1;
        data_type          member variable2;
        .................  .................
        data_type          member_variablen;
};
```

- Once structure_name is defined as a new data type, then variables of that type can be declared as:

  struct structure_name structure_variable;

- Eg: struct student s1,s2;

Example

```
struct student
{
        char name[20];
        int roll;
        char gender;
        char address[50];
        float fee, marks;
};
```

Defining a Structure

- We can combine both template declaration and structure variable declaration in one statement.
- struct student
  {
  char name[20];
  int roll;
  char gender;
  char address[50];
  float fee, marks;
  } s1,s2,s3;

Accessing Members of a Structure

- There are two types of operators to access members of a structure.
  - Member Operator (dot operator (.))
  - Structure pointer operator (->)
- For example: struct student

```
{
        char name[20];
        int roll;
         char section;
        float marks;
 };
```

- If we have declared structure variable as: struct student s1;
- Then, we can access each meber with the dot (.) operator:
  s1.name, s1.roll, s1.section, s1.marks;
- For declaration :struct student *s2;
- We can access each member using pointer: s2-> name, s2->roll, s2->secton

Structure Initialization

- Like any data type, a structure variable can be initialized.
- The initialization is shown in following example:

```
struct student
{
        char name[20];
        int roll;
       char section;
        float marks;
  } ;
```

- struct student s1 ={ "Raju",22,'A',55.5};

Example: 1

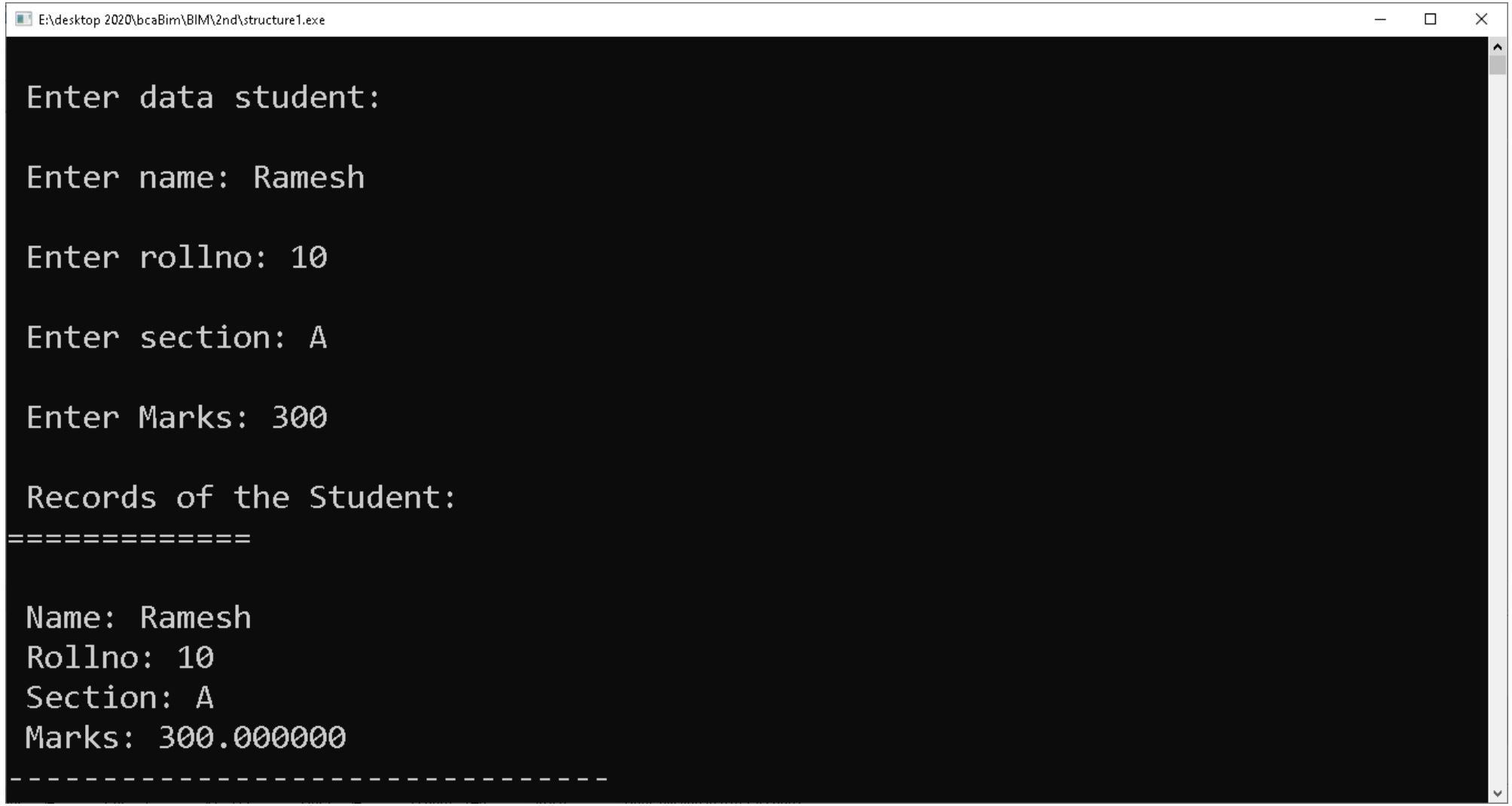
```
#include<stdio.h>
struct student
{
   char name[20];
  int roll;
  char section;
  float marks;
};

int main()
{
            struct student s;
            printf("\n Enter data student: \n");
            printf("\n Enter name: ");
            scanf("%s",s.name);
            printf("\n Enter rollno: ");
            scanf("%d", &s.roll);
            printf("\n Enter section: ");
            scanf(" %c", &s.section);
            printf("\n Enter Marks: ");
            scanf("%f", &s.marks);
            printf("\n Records of the Student:");
            printf("\n============\n");
            printf("\n Name: %s",s.name);
            printf("\n Rollno: %d",s.roll);
            printf("\n Section: %c",s.section);
            printf("\n Marks: %f",s.marks);

            return 0;

}
```

Output of above Program:



Array of Structures

- Like array of int, float or char type, thee may be array of structure.
- In this case, the array will have individual structure as its elements.
- For example, if we want to keep records of 50 students, then we have to make 50 structure variables s1, s2,....., s50. But this technique is not good.
- Instead we can create an array of structures to store those 50 records of students.
- Example: struct student s[50];

Example: Array of Structures

```
#include <stdio.h>
#include <conio.h>
struct student
{
        int roll;
        char name[20];
        int marks;
};
int main()
 {
 struct student st[5];
 int i;
 for(i=0;i<5;i++) {
  printf("Enter the Roll number of the student: ");
  scanf("%d",&st[i].roll);
  printf("Enter the name of the student: ");
  scanf("%s",st[i].name);
        printf("Enter the marks obtained : ");
        scanf("%d",&st[i].marks);
        }

 printf("\n ");
 printf("\n Students Records: \n");
 printf("======================\n");
 printf("Rollno \t Name \t Marks: \n");
 for(i=0;i<5;i++)
    printf("%d\t %s\t %d\n ", st[i].roll,st[i].name,st[i].marks);
getch();
 return 0;

}
```

Passing Structures to Function

- Whole structure can be passed to a function.
- In this call, only a copy of the structure is passed to the function, so that any changes done to the structure members are not reflected in the original structure.

Syntax for function call: function_name(structure_variable);

Syntax for function Definition:

Return_type function_name(struct tag_name structure_variable)

{

}

Example: Passing structure to function

```
#include<stdio.h>
void display(struct employee e);
struct employee
{
        char name[30];
        int id;
        float salary;
};

int main()
{
        struct employee e;
        printf("\n Enter employee name: ");
        scanf("%s", e.name);
        printf("\n Enter employee Id: ");
        scanf("%d", &e.id);
        printf("\n Enter salary: ");
        scanf("%f", &e.salary);
        display(e);

        return 0;
}
```

```
void display(struct employee e)
{
        printf("\n Name: %s", e.name);
        printf("\n Id:  %d", e.id);
        printf("\n Salary: %f  ", e.salary);
}
```

Nested Structure

- Nested structure in C is nothing but structure within structure.
- The feature of nesting one structure within another structure is used to create complex data types.
- One structure can be declared inside other structure as we declare structure members inside a structure.

```
struct address   {
  char city[20];
  int pin;
  char phone[14];
};

struct employee
{
    char name[20];
    struct address add;
};
```

```
Struct employee
{
     char name[20];

     struct address {
          char city[20];
          int pin;
          char phone[14];
      } add;
};
```

Example: nested Structure

```
#include <stdio.h>


struct student
  {
            int roll;
            char name[20];
            struct date
              {
                        int year;
                        int month;
                        int day;
              } dob;
};
```

```
int main()
{


            struct student st={2,"kapil",2052,11,19};
            printf("The Roll no: %d\n",st.roll);
            printf("Name: %s\n",st.name);
            printf("Date of birth: %d %d %d",st.dob.year,
                        st.dob.month, st.dob.day);
            getch();


}
```

Structure and Pointers

- The pointer is a variable that holds the address of another data variable.
- A pointer to structure means a pointer variable can hold the address of a structure.
- The address of structure variable can be obtained by using the '&' operator.
- All structure members inside the structure can be accessed using pointer, by assigning the structure variable address to the pointer.
- The pointer variable occupies 4 bytes of memory in 32-bit machine and 8 bytes in 64-bit machine.

Using Structure Pointer

- The declaration of a structure pointer is similar to the declaration of the structure variable. So, we can declare the structure pointer and variable inside and outside of the main() function.
- To declare a pointer variable in C, we use the asterisk (*) symbol before the variable's name.

```
struct structure_name *ptr;
```

- After defining the structure pointer, we need to initialize it, as the code is shown:

```
ptr = &structure_variable;
```

- We can also initialize a Structure Pointer directly during the declaration of a pointer.

```
struct  structure_name *ptr = &structure_variable;
```

- The members of the structure can be accessed using a special operator called as an arrow operator (->).

```
ptr->member
```

Example:1 Structure pointer

```
#include<stdio.h>
struct student{
  int sno;
  char sname[30];
  float marks;
};
main ( ){
  struct student s;
  struct student *st;
  printf("enter sno, sname, marks:");
  scanf ("%d%s%f", & s.sno, s.sname, &s. marks);
  st = &s;
  printf ("details of the student are");
  printf ("Number = %d\n", st ->sno);
  printf ("name = %s\n", st->sname);
  printf ("marks =%f\n", st ->marks);
  getch ( );
}
```

Example : 2

```
#include <stdio.h>
struct person
{
  int age;
  float weight;
};
int main()
{
   struct person *personPtr, person1;
   personPtr = &person1;
   printf("Enter age: ");
   scanf("%d", &personPtr->age);
   printf("Enter weight: ");
   scanf("%f", &personPtr->weight);
   printf("Displaying:\n");
   printf("Age: %d\n", personPtr->age);
   printf("weight: %f", personPtr->weight);
   return 0;
}
```

Unions

- Unions are similar to structure that are used to group a number of different variables together.
- Syntactically both structure and union are exactly same.
- The main difference between them is in storage.
- In structures, each member has its own memory location but all members of union use the dame memory location which is equal to the greatest member's size.
- Unions are used to conserve memory. Since same memory is shared by all members, one variable can reside into memory at a time. When another variable is set into memory, previous variable is replaced.

Defining Union

- The general syntax for declaring a union is:

```
union union_name
{
        data_type   member1;
        data_type   member2;
        ..........................
        data_type   member;
};
```

- Union variables are declared as: union student s1,s2,s3;
- We can also combine both template declaration and union variable in one statement.

Example

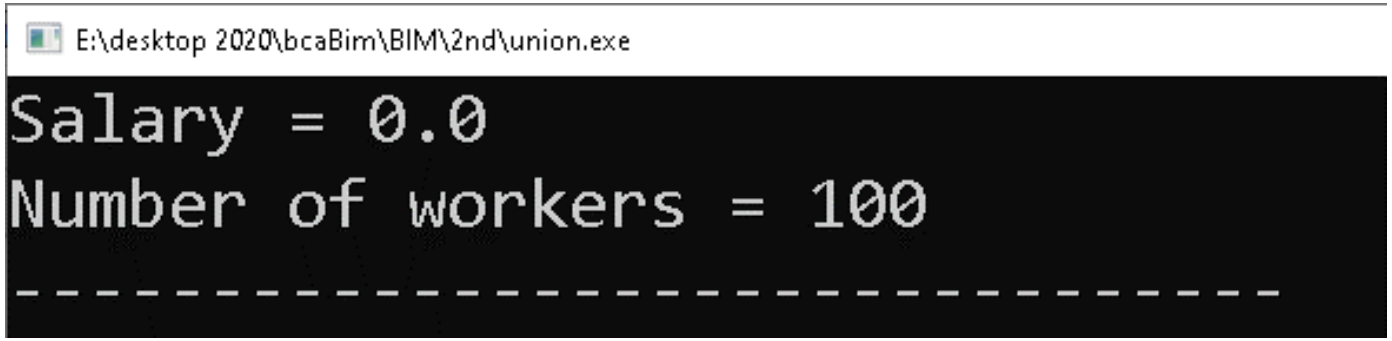
```
union student
{
        char name[10];
        int roll;
        char sec;
        float marks
};
```

Example:

```
#include <stdio.h>

union Job {
   float salary;
   int workerNo;
} j;
```

```
int main() {
   j.salary = 12.3;
   // when j.workerNo is assigned a value,
   // j.salary will no longer hold 12.3
   j.workerNo = 100;
   printf("Salary = %.1f\n", j.salary);
   printf("Number of workers = %d", j.workerNo);
   return 0;
}
```



Differences between Structure and Union

Structure	Union
1. The amount of memory required to store a structure variable is the sum of sizes of all the members.	1. In case of union, the amount of memory required is the same as member that occupies largest memory.
2. Each member with a structure is assigned its own unique address. So, it takes more memory than union.	2. All members within union share the same storage area of computer memory. So, it takes less memory than structure.
3. All the structure members can be accessed at any point of time.	3. Only one member of union can be accessed at any given time.

Thank You!